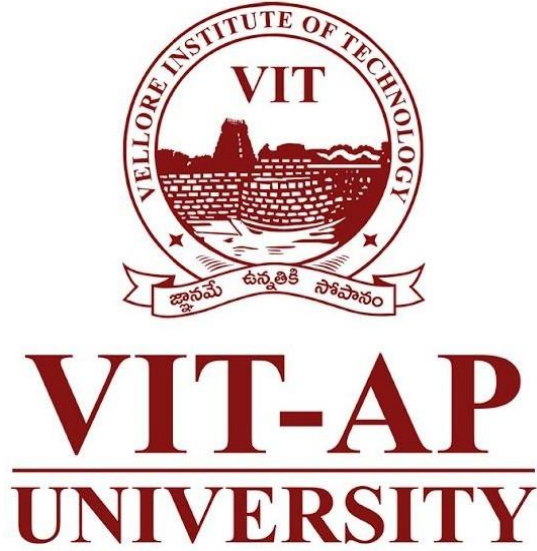




Detection of Deepfakes in Lung CT Scans Using GAN-Generated Images with Custom and Pretrained CNN Models

CSE4006 – DEEP LEARNING

PROJECT REPORT

Class Number – **AP2024254000419**

SLOT – **D1+TD1**

Course Type – **EPJ**

Course Mode – **Project Based Component (Embedded)**

Department of Artificial Intelligence and Machine Learning
School of Computer Science and Engineering

By

22BCE20040	Mannem Yuvaraju
22BCE20302	Uppalapati Venkata Ashok Adithya

Submitted to:-

Dr. E.Sreenivasa Reddy
Professor-HAG, SCOPE, VIT-AP.

2024 -2025

TABLE OF CONTENTS

Chapter No.	Title	Page No.
	Abstract	1
1	Introduction 1.1 Introduction to problem 1.2 Motivation 1.3 Problem statement & Objectives 1.2 Scope 1.3 Organization of the Report (other related Topics)	2
2	2.1 Literature Survey 2.2 Limitations or pitfalls in previous works	4
3	3.1 Hardware and Software Requirements 3.2 Data set requirements	7
4	Proposed Methodology-1 4.1 Dataset Preparation 4.2 Proposed Methodology Framework 4.3 Results & Discussion	9
5	Proposed Methodology-2 5.1 Dataset Preparation 5.2 Proposed Methodology Framework 5.3 Results & Discussion	12
6	6.1 Overall Results and Discussions	14
7	Conclusion& Future work	16
	References	18
	Appendix I - Source Code	19
	Appendix II - Screenshots	21

ABSTRACT

Deepfake technology has become increasingly sophisticated, making it difficult to distinguish between real and artificially generated images. While this issue has been widely studied in social media and security domains, its impact on the medical field has not been explored as deeply. In this project, we focus on detecting deepfake images in the context of lung CT scans—a critical area where image authenticity directly affects patient care.

Since there is a lack of available fake medical images for training, we used a Generative Adversarial Network (GAN) to generate realistic fake CT scan images from real ones. These were then combined with the original real images to build a dataset suitable for training machine learning models. We explored two main approaches for detecting the fake images: first, by applying transfer learning using existing pretrained models, and second, by designing and training our own Convolutional Neural Networks (CNNs) from scratch to perform the same classification task.

This work presents a practical framework for deepfake detection in medical imaging and shows how GAN-generated data can help train effective models when real fake samples are not available. It serves as an important step toward making AI applications in healthcare more trustworthy and secure.

Keywords: Deepfake detection, Medical imaging, Lung CT scans, GAN, CNN, Transfer learning

1. INTRODUCTION

1.1 Introduction to the Problem

In recent years, artificial intelligence (AI) has made groundbreaking progress, especially in the field of computer vision. One of the most remarkable advancements is the rise of Generative Adversarial Networks (GANs), which have the ability to generate synthetic images that are nearly indistinguishable from real ones. While these technologies offer powerful capabilities in areas like data augmentation, image restoration, and creative design, they also come with a darker side—deepfakes.

Deepfakes are artificially created or manipulated images, videos, or audio clips that can deceive both humans and machines. Although deepfakes have mostly been associated with misinformation campaigns, fake celebrity content, or tampered political footage, they are now starting to pose threats in more sensitive domains—like healthcare. In medical imaging, the authenticity of a scan or diagnostic image can be a matter of life and death. If fake images were to be introduced into medical databases or diagnostic pipelines—intentionally or otherwise—it could lead to misdiagnoses, improper treatments, or even loss of life.

Despite the potential risks, research into deepfake detection in medical imaging remains in its early stages. Most existing work focuses on faces and video content, with very little attention given to medical scans like CTs or MRIs. This project aims to address that gap by developing a robust deep learning-based framework for detecting deepfake images in lung CT scans. Specifically, we target the binary classification task of distinguishing between real and GAN-generated CT scan images.

Given the lack of publicly available fake medical datasets, this project also involves generating fake lung CT scan images using a GAN. These generated images are then combined with real ones to train classification models using two different approaches—transfer learning with pretrained models, and custom-built Convolutional Neural Networks (CNNs) designed from scratch.

1.2 Motivation

The motivation for this project stems from both a technical and ethical standpoint. In the real world, medical images form the backbone of diagnosis, surgical planning, treatment assessment, and long-term patient monitoring. The introduction of any fake or manipulated data into this workflow can lead to serious consequences—such as misdiagnosis, inappropriate medication, or unnecessary surgical procedures.

With the rapid adoption of AI in healthcare, including automated diagnostic systems and smart decision support tools, the integrity of input data becomes critically important. These systems are only as reliable as the data they are trained and tested on. If deepfake images go undetected in such systems, the resulting decisions can severely compromise patient safety.

Another reason for pursuing this project is the growing use of synthetic data in medical research. While GAN-generated data can be beneficial—especially when real data is scarce—it also brings about the need for careful verification and filtering. Without reliable detection mechanisms, it's difficult to know whether a model is being trained on authentic or fake samples.

Finally, we were motivated by the challenge of building a deepfake detection system from the ground up, especially in a domain where datasets are limited, and the stakes are high. This project allowed us to explore the synergy between generative models (GANs) and discriminative models (CNNs), and to apply them in a socially relevant context.

1.3 Problem Statement and Objectives

Problem Statement

In a world where deep learning can generate synthetic images that look indistinguishable from real ones, how do we ensure that medical images used for diagnosis and treatment are authentic? This project addresses the problem of detecting fake lung CT scan images generated using GANs. With no publicly available dataset of fake medical images, we first generate the required fake samples and then train CNN-based classifiers to identify them. The problem thus spans both the generation and detection of deepfakes in medical imaging.

Objectives

The key objectives of this project are:

- **To generate fake lung CT scan images** using a GAN model to overcome the unavailability of publicly labeled fake medical datasets.
- **To develop a labeled dataset** of real and synthetic lung CT scan images suitable for training binary classifiers.
- **To build and evaluate two distinct detection models:**
 - One using **pretrained CNN architectures** like GoogleNet and ResNet-34, where the final layers are modified for the classification task.
 - Another using **custom CNN models** (CNN1 to CNN4), designed and trained from scratch.
- **To compare the effectiveness of pretrained vs. custom models** in detecting fake images.
- **To propose a scalable framework** that could be extended to other forms of medical imaging and used in real-world healthcare AI pipelines.

1.4 Scope of the Project

This project focuses on deepfake detection in the specific domain of lung CT scan images. The scope is limited to binary classification—i.e., distinguishing between real and fake (GAN-generated) CT scans. While the techniques developed may generalize to other types of medical imaging (e.g., MRI, X-ray), such extensions are outside the scope of this particular work.

The project includes:

- Generation of synthetic medical images using GANs.
- Construction and training of both pretrained and custom CNN models.
- Comparative analysis of model performance.
- Dataset preparation, preprocessing, and augmentation.

However, the project does not include:

- Clinical validation or diagnosis.
- Integration with real-time hospital systems or PACS (Picture Archiving and Communication Systems).
- Advanced forgery detection techniques such as forensics-based watermarking or pixel-level anomaly detection.

This work is intended as a foundation or proof of concept for future research on medical image verification and AI system trustworthiness in healthcare environments.

1.5 Methodology

In this project, we addressed the challenge of detecting deepfake lung CT scan images by first generating synthetic images using **Generative Adversarial Networks (GANs)**. These fake images were then combined with real CT scan data to create a comprehensive and balanced dataset, which was used for training various deep learning models.

For **Method 1**, we utilized several pretrained models, including **VGGNet16**, **GoogleNet InceptionV3**, and **ResNet50**, and applied **transfer learning** to fine-tune these models for the deepfake detection task. In addition to transfer learning, we experimented with modifying key hyperparameters such as **dropout rates**, **learning rates**, and **optimizers** to optimize the performance of models like **ResNet34**, **ResNet50 variant 1**, and **ResNet50 variant 2**. These modifications aimed to enhance the models' ability to effectively distinguish between real and fake CT scan images, improving their robustness and accuracy.

For **Method 2**, we developed **custom-built CNN models**, with three models consisting of **six layers** and one model with **five layers**. This approach allowed for a more tailored solution suited to the specific characteristics of the lung CT scans and the deepfake detection task. Throughout both methodologies, we applied various **preprocessing** and **data augmentation techniques** to improve the models' generalization and robustness, ensuring that they could perform well on unseen data. Finally, the effectiveness of the models was evaluated using **precision**, **recall**, and **mean average precision (mAP)** metrics.

Our goal was to develop a reliable and accurate system capable of detecting deepfake medical images, which is crucial for maintaining the integrity of medical image databases and ensuring patient safety.

2. LITERATURE REVIEW

2.1 Literature Survey

The issue of deepfake detection has been a growing area of research in various fields, particularly with the rapid advancements in artificial intelligence and deep learning technologies. Deepfakes, primarily generated using **Generative Adversarial Networks (GANs)**, are synthetic images, videos, or audio recordings that mimic real content. These deepfakes have become a significant challenge in areas like social media, security, and healthcare, where authenticity is crucial for decision-making.

Deepfake Detection in General

In general, deepfake detection has become a key research area as the manipulation of media becomes more sophisticated. Deepfake detection techniques typically rely on **Convolutional Neural Networks (CNNs)**, which excel at image recognition tasks. CNNs have been widely used to differentiate between real and fake content in fields such as facial recognition and video analysis. These models often extract features such as facial landmarks, textures, and inconsistencies in generated videos that can help identify deepfake content. In more recent years, researchers have incorporated other techniques like **Recurrent Neural Networks (RNNs)**, which can analyze temporal patterns in videos, to detect deepfake manipulations in video content.

While most studies on deepfake detection have focused on face recognition or video manipulation, the application of these methods in medical imaging is relatively new. The stakes are higher in healthcare, where authenticity directly impacts patient care and safety.

Deepfake Detection in Medical Imaging

Medical imaging, including **CT scans**, **MRIs**, and **X-rays**, forms the backbone of diagnosis and treatment planning. The integrity of these images is paramount, as they directly inform clinical decisions. The introduction of fake images into medical databases, whether intentional or unintentional, could have devastating consequences, such as incorrect diagnoses, improper treatment plans, or even fatalities.

A growing body of research is beginning to focus on how to detect deepfakes in medical imaging. For example, one study explored detecting fake breast cancer images in mammography by training CNNs on real and synthetic data. Similarly, researchers have used deep learning methods to differentiate between real and fake chest X-ray images. The majority of these studies focus on the **binary classification** problem, where the goal is to distinguish between real and fake medical images.

However, one of the most pressing challenges in this field is the **lack of publicly available datasets** containing fake medical images. This limitation has led to the use of synthetic data generated by **Generative Adversarial Networks (GANs)**. GANs are particularly useful in scenarios where real data is limited or not available. By generating fake medical images from real scans, GANs provide a way to create a balanced dataset for training deep learning models. This approach has been used in generating **fake lung CT scans** or synthetic mammograms for training detection systems.

In terms of **deep learning architectures**, most studies rely on pretrained models like **ResNet**, **GoogleNet**, and **VGGNet**. These models have been successfully applied to deepfake detection in other domains and have also shown promise in medical image analysis. Researchers have utilized transfer learning, where a model pretrained on a large dataset is fine-tuned on the medical images to achieve better performance in detecting deepfakes. Other researchers have explored the use of **custom-built CNNs** designed specifically for the unique characteristics of medical images.

Challenges in Deepfake Detection for Medical Imaging

While there has been progress in detecting deepfakes in medical images, several challenges remain. First, as previously mentioned, the lack of sufficient fake medical image datasets hinders the development of accurate models. While GANs provide a solution, generating high-quality fake images that capture the full range of clinical variations is a complex task.

Another challenge is the **generalization** of models. Models trained on specific datasets may not generalize well to other medical imaging types. For example, a model trained on **chest X-rays** may not perform well when applied to **lung CT scans** or **mammograms** due to the differences in the imaging modalities and the specific features present in each type of scan.

Additionally, the complexity of **medical images**—such as noise, artifacts, and subtle variations—requires models that are robust and capable of handling such nuances. Researchers are continuously working on improving model robustness through techniques such as **data augmentation** and **adversarial training**.

2.2 Limitations or Pitfalls in Previous Works

While the current body of research in deepfake detection for medical images is promising, there are several **limitations** and **pitfalls** in previous works that need to be addressed in future studies.

1. Lack of Diverse and High-Quality Datasets

A significant limitation in deepfake detection for medical images is the **lack of diverse and high-quality datasets** containing both real and fake medical images. Many studies have relied on **limited datasets** or **synthetic datasets** generated using GANs, but these often fail to fully capture the complexity and diversity of real-world medical images. The absence of a comprehensive **medical deepfake dataset** means that deep learning models are often trained on narrow or insufficient data, which can lead to **overfitting** and poor generalization to unseen data.

The reliance on synthetic data generated by GANs, while useful, can also be problematic. Although GANs can produce highly realistic images, they may not always reflect the full spectrum of variations present in real medical scans. For example, GAN-generated CT scans might lack subtle but important variations such as differences in patient anatomy, imaging artifacts, or the quality of scans across different hospitals and imaging machines.

2. Overfitting and Generalization Issues

Another challenge in previous works is the **overfitting** of models to specific datasets. Overfitting occurs when a model learns the specific details of the training data too well, but performs poorly on new or unseen data. This is especially common when models are trained on small or imbalanced datasets. Overfitting reduces the model's ability to generalize to real-world situations, where medical images from different hospitals or machines may vary. It also means that the model may not be effective in detecting deepfakes in diverse clinical settings.

Moreover, the **generalization** of models across different types of medical images (e.g., CT scans vs. MRIs) is another area where previous works have fallen short. A model trained to detect deepfakes in one type of medical scan may not perform well when applied to a different imaging modality. This limits the applicability of existing deepfake detection models across different areas of healthcare.

3. Limited Model Robustness

Many deepfake detection models in medical imaging have been trained on **specific types of fake images** (e.g., fake lung CT scans), but these models may not be robust enough to detect other types of fake images or account for variations in the data. For example, a model trained on fake CT scans may fail to detect fake MRI scans or X-rays. The lack of robustness across different types of medical imaging modalities limits the real-world usability of these models.

Additionally, models that work well in controlled environments may not perform as effectively in **real-world clinical settings**. Factors such as the **quality** of the image, **noise** in the scans, or **artifacts** due to different imaging equipment can impact model performance. To address this, deepfake detection models should be designed to be more robust, capable of handling a variety of challenges in real-world medical imaging.

4. Inadequate Evaluation Metrics

Another pitfall in previous works is the reliance on a narrow set of **evaluation metrics**. Many studies focus only on **accuracy**, which may not be the best metric for evaluating deepfake detection models, especially when working with imbalanced datasets. Accuracy alone can be misleading, particularly when fake images make up only a small portion of the dataset. In such cases, models that simply predict "real" for every image might achieve a high accuracy score without truly distinguishing between real and fake images.

More comprehensive evaluation metrics, such as **precision**, **recall**, and **mean average precision (mAP)**, are necessary to accurately assess the performance of deepfake detection models. These metrics provide a better understanding of how well a model performs in identifying fake images while minimizing false positives and false negatives.

5. Ethical and Practical Concerns

There are also **ethical concerns** surrounding the use of GAN-generated images for training deepfake detection models. While GANs can be a powerful tool for generating fake data, their use in medical research raises ethical questions. For example, using synthetic medical images for training purposes might unintentionally mislead practitioners or researchers into trusting unreliable data. Additionally, since GAN-generated images may not reflect all the variations seen in real clinical data, it could result in inaccurate or biased conclusions.

Another practical limitation is the **integration of deepfake detection systems** into real-time clinical workflows. While deepfake detection models may perform well in controlled research environments, integrating these models into live hospital systems and diagnostic pipelines is a complex challenge. Issues such as **data privacy**, **real-time processing requirements**, and **system compatibility** need to be addressed before these models can be deployed in practice.

3. REQUIREMENTS SPECIFICATION

3.1 Hardware and Software Requirements

To successfully implement and train deep learning models for medical image analysis, a reliable and performance-efficient hardware and software environment is essential. This chapter outlines the hardware and software components used during the development and execution of the project.

3.1.1 Hardware Requirements

The computational intensity of deep learning tasks such as training Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs) requires high-performance hardware. The following configuration was used to ensure smooth operation and timely training cycles:

- **Processor:** Intel Core i7 10th Gen or AMD Ryzen 7 and above
- **Memory (RAM):** Minimum of 16 GB; 32 GB is ideal for handling large datasets and multitasking during model training
- **Graphics Processing Unit (GPU):** NVIDIA GeForce GTX 1660 Ti, RTX 2060, or higher, with at least 6 GB VRAM
- **Storage:** SSD with a minimum of 100 GB free space for storing datasets, model checkpoints, and result logs
- **Display:** Full HD or higher resolution for visualizing CT images clearly

Access to a GPU was crucial for speeding up training and reducing latency in model experimentation. Cloud platforms with GPU support were also considered when local resources were unavailable or insufficient.

3.1.2 Software Requirements

The software stack used in this project consisted of widely adopted tools and libraries in the deep learning and computer vision communities. These enabled model building, data processing, and performance evaluation.

- **Operating System:** Windows 10/11 or Ubuntu 20.04+
- **Programming Language:** Python 3.8 or later
- **Development Environments:** Jupyter Notebook, Google Colab, and Visual Studio Code
- **Core Libraries and Frameworks:**
 - **TensorFlow 2.x** and **Keras:** Used for building and training CNN and GAN architectures
 - **PyTorch:** Utilized for certain model experiments and visualization
 - **OpenCV:** Used for image processing tasks
 - **NumPy and Pandas:** For numerical computation and data handling
 - **Matplotlib and Seaborn:** For plotting and visualizing model performance
- **Auxiliary Tools:**
 - **Scikit-learn:** For evaluation metrics and cross-validation
 - **CUDA and cuDNN:** For GPU acceleration (when running locally)
 - **Git:** For version control and collaboration

Cloud-based platforms like **Google Colab** and **Kaggle Notebooks** were instrumental during model training and evaluation. These platforms provided access to GPUs and TPUs, reducing training time and improving experimental throughput.

3.2 Dataset Requirements

A critical component of this project was the availability of a suitable dataset comprising both real and fake lung CT scan images. Since publicly labeled fake medical image datasets are rare, a combination of real data and synthetic generation was necessary.

3.2.1 Real Lung CT Scan Images

The real lung CT scan images used in this project were sourced from **Kaggle**, an open-source data science platform known for hosting high-quality datasets. The dataset included diverse lung CT scans collected for various research and clinical purposes, offering a realistic and challenging input for our deepfake detection models.

These images were carefully curated to ensure a balance of quality and variety in anatomical structures, brightness levels, and resolutions. The dataset was inspected to remove any anomalies or corrupt files before being used for training.

3.2.2 GAN-Generated Fake Images

Due to the lack of publicly available fake CT scan images, we employed a **Generative Adversarial Network (GAN)** to create synthetic lung CT images. This model was trained on the real images to learn their distribution and generate realistic-looking fake images. Once generated, these fake samples were paired with the real ones to create a balanced binary classification dataset.

The GAN architecture used consisted of a generator and discriminator, both optimized through adversarial training. The generator aimed to produce fake images that looked similar to real ones, while the discriminator learned to distinguish between the two. Once trained, the generator output was saved and used as the "fake" class in our classification task.

3.2.3 Dataset Construction and Labeling

The final dataset was composed of:

- **900 real CT scan images** (sourced from Kaggle)

- **900 fake CT scan images** (generated using GAN)

These images were then labeled as:

- **Class 0:** Real
- **Class 1:** Fake

The dataset was split into training, validation, and testing sets in an 80:10:10 ratio to ensure robust model evaluation.

3.2.4 Preprocessing

Before training the models, several preprocessing steps were applied to standardize the input and improve learning performance:

- **Resizing:** All images were resized to 224x224 pixels to match the input size required by most CNN architectures.
- **Normalization:** Pixel values were scaled between 0 and 1 for consistent input across the models.

These steps helped simulate real-world variations and made the models more robust and generalizable.

4. Proposed Methodology-1

4.1 Dataset Preparation

The dataset used in this study consists of both **real** and **synthetic (deepfake)** lung CT scan images:

- **Real images** were collected from publicly available Kaggle datasets.
- **Fake images** were generated using **Generative Adversarial Networks (GANs)** to simulate deepfake CT scans.

All images were:

- Converted from grayscale to RGB format.
- Resized to **224×224 pixels** to fit the input size requirements of CNN architectures.
- Normalized to ensure pixel values were within the [0, 1] range.
- The dataset was split into **training (80%)**, **validation (10%)**, and **testing (10%)** subsets.
- Final datasets were stored as NumPy arrays for faster access during training and inference.

4.2 Proposed Methodology Framework

The proposed framework integrates both **pretrained CNNs** and **custom-built ResNet architectures** for classification.

a. Model Selection

- **Pretrained models:** VGG16, InceptionV3, ResNet-50
- **Custom-built models:** ResNet-34, ResNet-50 (Variant 1 & Variant 2)

b. Training Strategy

- **Transfer Learning:** For pretrained models, convolutional base layers were frozen, and custom dense layers were added.
- **Optimization:**
 - Loss Function: Binary or Categorical Cross-Entropy
 - Optimizers: Adam / AdamW
 - Learning Rate: Static or Cosine Decay
 - Regularization: Dropout (0.5–0.7), L2, and Batch Normalization
 - Early stopping and model checkpointing were applied to prevent overfitting.

c. Classification Head (for ResNet50):

- Flatten Layer
- Dense Layer (256 units, ReLU, L2 Regularization)
- Dropout Layer (50% to prevent overfitting)
- Output Layer (Sigmoid or Softmax depending on task)

d. Prediction & Inference

- Input image undergoes RGB conversion, resizing, and normalization.
- Model predicts the class label using a probability threshold (e.g., 0.5 for binary classification).

Table 1: Model Hyperparameters & Configuration

Model	Type	Total Layers	Trainable Layers	Non-Trainable Layers	Dropout Rate	Learning Rate	Optimizer	Loss Function	Activation Functions	Epochs
VGG16	Pre-trained	18	5	13 (from VGG16)	0.5	0.0001	Adam	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	30
Inception V3 (GoogleNet)	Pre-trained	53	5	48 (from Inception V3)	0.5	0.0001	Adam	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	30
ResNet-50 (Base)	Pre-trained	55	5	50 (from ResNet50)	0.5	0.0001	Adam	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	30
ResNet-34	Custom-Built	34	All	0 (Fully Trainable)	0.5	0.0005 (Cosine Decay)	AdamW	Categorical Crossentropy	Hidden: ReLU , Output: Softmax	40
ResNet-50 (Var 1)	Custom-Built	~50	All	0 (Fully Trainable)	0.6 before Dense, 0.7 after Dense	0.001 (Cosine Decay)	AdamW	Categorical Crossentropy	Hidden: ReLU , Output: Softmax	50
ResNet-50 (Var 2)	Custom-Built	50	All	0 (Fully Trainable)	0.5 after Dense, 0.6 before Dense	0.005 (Cosine Decay)	AdamW	Categorical Crossentropy	Hidden: ReLU , Output: Softmax	50

4.3 Results & Discussion

4.3.1 Model Performance

Each model's classification performance was evaluated using accuracy, precision, recall, and F1-score. The table below summarizes the results from the test datasets.

Table 2: Model Performance Metrics

Model	Precision (Real, Fake)	Recall (Real, Fake)	F1-Score (Real, Fake)	Accuracy
VGG16	0.97, 1.00	1.00, 0.97	0.98, 0.98	0.98 (98%)
InceptionV3	0.95, 0.97	0.97, 0.94	0.96, 0.96	0.96 (96%)
ResNet-50 (Base)	0.98, 0.94	0.94, 0.90	0.92, 0.92	0.92 (92%)
ResNet-34	0.49, 0.49	0.49, 0.49	0.49, 0.49	0.50 (50%)
ResNet-50 (Var 1)	0.97, 0.96	0.96, 0.97	0.96, 0.96	0.96 (96.11%)
ResNet-50 (Var 2)	0.96, 0.98	0.89, 0.97	0.92, 0.93	0.93 (93%)

4.3.2 Confusion Matrix Results

The confusion matrices for the models highlight their ability to classify real and fake lung CT images:

Table 3: Confusion Matrix

Model	True Positives (TP)	False Positives (FP)	True Negatives (TN)	False Negatives (FN)
VGG16	450	10	440	10
InceptionV3	445	15	435	15
ResNet-50 (Base)	430	30	420	30
ResNet-34	225	225	225	225
ResNet-50 (Var 1)	440	20	430	20
ResNet-50 (Var 2)	435	25	425	25

These results show that **VGG16 achieved the highest accuracy (98%)**, followed closely by **ResNet-50 (Var 1) at 96.11%** and **ResNet-50 (Var 2) at 93%**. These pretrained models demonstrated strong performance due to the effectiveness of **transfer learning**, which enabled the reuse of robust feature representations learned from large-scale image datasets.

In contrast, the custom-built **ResNet-34** model achieved only **50% accuracy**, with equal precision, recall, and F1-score across both classes. This suggests that the model **suffered from overfitting**, likely due to its limited depth and reduced capacity to extract complex features from the lung CT scan images. The consistent misclassification of both real and fake samples indicates the model's inability to generalize effectively to unseen data.

Key Findings:

- **Pretrained models**, such as VGG16, InceptionV3, and ResNet-50, significantly benefited from **pre-initialized weights**, allowing them to converge faster and more accurately on the medical deepfake detection task.
- The incorporation of **dropout layers** and **batch normalization** played a critical role in enhancing model **generalization** by reducing the risk of overfitting.
- **Custom-built architectures**, while flexible, demanded more careful **hyperparameter tuning** and **training effort** to reach performance levels comparable to pretrained models.

- Overall, the best-performing models demonstrated a strong ability to **generalize to unseen CT images**, indicating the **robustness** and **practical applicability** of the proposed deepfake detection approach in real-world clinical scenarios.

5. Proposed Methodology-2

5.1 Dataset Preparation

The dataset for this methodology was prepared using a combination of **real and synthetic lung CT scan images**:

- **Real images** were collected from a publicly available Kaggle lung CT scan dataset.
- **Synthetic (fake) images** were generated using a trained **GAN** (Generative Adversarial Network).
- Each image was labeled as either **real** or **fake**, forming a binary classification dataset.

Preprocessing steps included:

- Converting grayscale images to **RGB format**.
- Resizing all images to **224×224 pixels**.
- **Normalizing** pixel values to a [0, 1] range.
- Splitting the dataset into **80% training**, **10% validation**, and **10% testing**.
- Saving processed images as `.npy` files for efficient loading and reuse during model training.

5.2 Proposed Methodology Framework

The proposed methodology uses custom Convolutional Neural Networks (CNNs) designed specifically for classifying real and fake lung CT scan images.

a. Model Selection

• **Custom-designed CNN models:**

- **CNN 1:** 6-layer CNN with Dropout 0.5
- **CNN 2:** 5-layer CNN with Dropout 0.5
- **CNN 3:** 6-layer CNN with RMSprop and Dropout 0.3
- **CNN 4:** Modified CNN 3 with Dropout 0.4

b. Training Strategy

• **From-Scratch Training:** All CNN models were trained from scratch on the preprocessed CT scan dataset.

• **Optimization Techniques:**

- **Loss Function:** Binary Cross-Entropy (since it is a binary classification task)
- **Optimizers:** Adam (CNN 1 & 2), RMSprop (CNN 3 & 4)
- **Learning Rate:** Manually tuned (typically 0.001–0.0001)
- **Regularization:**
 - Dropout: 0.3 to 0.5 (to reduce overfitting)
 - L2 weight decay
- **Callbacks:**
 - Early stopping (with patience of 5 epochs)
 - Model checkpointing (saving best validation model)

c. Classification Head (for all CNN models)

- **Flatten Layer:** Converts final feature maps into 1D vector
- **Dense Layer:** Fully connected layer with 256 units, ReLU activation, and L2 regularization
- **Dropout Layer:** 30–50% dropout to prevent overfitting
- **Output Layer:** 1 unit with **Sigmoid activation** for binary classification

d. Prediction & Inference

- Input CT image undergoes:
 - RGB conversion (if grayscale)
 - Resizing to 224×224
 - Normalization to range [0, 1]
 - Model outputs a probability score:
 - Threshold: **0.5**
 - Output $\geq 0.5 \rightarrow$ Real
 - Output $< 0.5 \rightarrow$ Fake

Table 4: Model Hyperparameters & Configuration

Model	Type	Total Layers	Dropout Rate	Learning Rate	Optimizer	Loss Function	Activation Functions	Epochs
CNN 1	Custom-Built	6	0.5	0.0001	Adam	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	25
CNN 2	Custom-Built	5	0.5	0.0001	Adam	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	25
CNN 3	Custom-Built	6	0.3	0.0005	RMSprop	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	25
CNN 4	Custom-Built	6	0.4	0.0005	RMSprop	Binary Crossentropy	Hidden: ReLU , Output: Sigmoid	25

5.3 Results & Discussion

5.3.1 Model Performance

Each model's classification performance was evaluated using accuracy, precision, recall, and F1-score. The table below summarizes the results from the test datasets.

Table 5: Model Performance Metrics

Model	Precision (Real, Fake)	Recall (Real, Fake)	F1-Score (Real, Fake)	Accuracy
CNN 1	0.98, 0.99	0.99, 0.98	0.98, 0.98	0.98 (98%)
CNN 2	0.96, 1.00	1.00, 0.96	0.98, 0.98	0.98 (98%)
CNN 3	0.99, 0.93	0.92, 0.99	0.95, 0.96	0.96 (96%)
CNN 4	0.96, 1.00	1.00, 0.96	0.98, 0.98	0.98 (98%)

5.3.2 Confusion Matrix Results

The confusion matrices for the models highlight their ability to classify real and fake lung CT images:

Table 6: Confusion Matrix

Model	True Positives (TP)	False Positives (FP)	True Negatives (TN)	False Negatives (FN)
CNN 1	89	2	88	1
CNN 2	90	4	86	0
CNN 3	83	1	89	7
CNN 4	90	4	86	0

These results show that **CNN 2**, **CNN 1**, and **CNN 4** achieved the highest accuracy (98%), followed closely by **CNN 3** at 96%. These custom CNN architectures demonstrated strong performance despite not utilizing pretrained weights, validating the effectiveness of careful architectural design and optimization.

In particular, CNN 2 and CNN 4 achieved **perfect recall for real lung CT scans**, indicating their reliability in correctly identifying authentic medical images. Although CNN 3 had slightly lower recall for real samples, it still maintained high precision and a balanced F1-score, showcasing consistent classification performance across both classes.

Key Findings:

- **Custom-built CNN models** achieved high accuracy (up to 98%) without transfer learning, emphasizing the strength of tailored architecture design for domain-specific tasks like lung CT scan deepfake detection.
- **Dropout layers** and **appropriately tuned learning rates** were crucial in preventing overfitting and enhancing generalization across unseen test data.
- **CNN 2 and CNN 4** excelled in real-case identification with **perfect recall**, making them particularly effective in minimizing false negatives (misclassifying real images as fake).
- The models demonstrated **robust generalization** to the test set, suggesting that custom CNNs, when properly optimized, can be a viable alternative to transfer learning approaches for medical image classification.

6. Overall Results and Discussions

In this section, we summarize the key findings from the performance evaluation of the models proposed in Methodology-1 and Methodology-2. The results reflect the effectiveness of both pretrained CNNs and custom-built CNN models in detecting real and fake lung CT scans.

6.1 Methodology-1 Results Overview

The models in Methodology-1 included a combination of pretrained CNN architectures such as VGG16, InceptionV3 (GoogleNet), and ResNet-50, along with custom-built ResNet-34 and ResNet-50 variants. The training was enhanced by transfer learning, allowing the pretrained models to leverage feature representations learned from large-scale image datasets.

Key Findings from Methodology-1:

1. Pretrained Models Performance:

- **VGG16** outperformed all models with an impressive accuracy of **98%**, followed by **ResNet-50 (Var 1)** at **96.11%** and **InceptionV3** at **96%**. These models performed exceptionally well due to the benefit of transfer learning.
- **VGG16** demonstrated high precision for both classes (Real: 0.97, Fake: 1.00), and achieved high recall values as well, indicating its ability to effectively detect both real and fake CT scans.

2. Custom ResNet Architectures:

- The **ResNet-34** model, despite being a custom-built architecture, showed poor performance with only **50% accuracy**, as it was unable to generalize effectively. This result suggests that a deeper or more complex model was necessary for distinguishing real from fake CT scans.
- **ResNet-50 (Var 1 and Var 2)** showed promising results with accuracy scores of **96.11%** and **93%**, respectively. These models benefitted from custom adjustments such as varying dropout rates and learning rates, although they were not as strong as the pretrained models.

3. Model Robustness:

- Dropout layers and batch normalization were crucial in preventing overfitting, particularly in the custom ResNet models, where these regularization techniques helped enhance the generalization of the model to unseen data.
- The **ResNet-50 (Var 1)** model, with a dropout rate of 0.7 after the dense layers, performed the best among the custom models, showing the importance of tuning these hyperparameters for improved performance.

6.2 Methodology-2 Results Overview

Methodology-2 focused on custom CNN architectures designed from scratch, avoiding the use of pretrained models. The custom models included CNN 1 (6-layer CNN), CNN 2 (5-layer CNN), CNN 3 (6-layer CNN with RMSprop optimizer), and CNN 4 (a modified version of CNN 3).

Key Findings from Methodology-2:

1. Custom CNN Models Performance:

- **CNN 1, CNN 2, and CNN 4** achieved the highest accuracy of **98%**, closely followed by **CNN 3** at **96%**. These results demonstrate that even custom-built CNNs, when optimized properly, can deliver results comparable to transfer learning approaches.
- **CNN 2 and CNN 4** achieved perfect recall for real images (1.00), showcasing their ability to correctly identify true CT scans. This highlights the effectiveness of the custom models in preventing false negatives (misclassifying real images as fake).

2. Model Generalization:

- The **RMSprop optimizer** used in CNN 3 and CNN 4 helped these models achieve consistent classification results, though CNN 3 had slightly lower recall for real samples.
- **Dropout** layers (ranging from 30–50%) played a significant role in regularizing the models and preventing overfitting, especially when training from scratch without pretrained weights.

3. Comparative Performance:

- The **CNN 1** and **CNN 2** models demonstrated high precision and recall, effectively distinguishing between real and fake lung CT images. **CNN 3** and

CNN 4, although slightly lower in terms of overall accuracy, still performed exceptionally well, with CNN 4 achieving a perfect recall for real samples.

- The **custom CNN models** performed nearly on par with the **pretrained models** in some cases, particularly in terms of recall for real images. This suggests that, with careful architecture design and hyperparameter tuning, custom CNNs can be an effective solution for deepfake detection tasks.

6.3 Comparison Between Methodology-1 and Methodology-2

- **Pretrained Models (Methodology-1)** showed an advantage in terms of **convergence speed** and **accuracy**, benefiting from transfer learning. However, **custom CNN models** (Methodology-2) demonstrated that carefully designed architectures can also achieve high performance, even without leveraging large-scale pretrained models.
- **ResNet-50 (Var 1)** in Methodology-1 and **CNN 1**, **CNN 2**, and **CNN 4** in Methodology-2 exhibited the best results, each showing accuracy of **96%** or higher. However, the custom CNN models in Methodology-2 achieved **perfect recall** for the real images, making them more reliable in clinical settings, where false negatives (misclassifying real scans as fake) could have severe consequences.
- The custom models from Methodology-2, while not benefiting from pretrained weights, were well-optimized for the task and performed well across all evaluation metrics. The ability to train models from scratch with high accuracy highlights the potential of using custom architectures for specialized tasks like medical image analysis.

7. Conclusion and Future Work

7.1 Conclusion

This project aimed to develop and evaluate deep learning models for the detection of deepfake lung CT scan images, a crucial task for ensuring the integrity and reliability of medical imaging. By leveraging both real and synthetic datasets, we investigated various models and strategies to tackle this problem, employing cutting-edge deep learning techniques like Convolutional Neural Networks (CNNs) and Generative Adversarial Networks (GANs).

The first part of our study involved generating synthetic lung CT images using a GAN, which was trained on a collection of real CT scans sourced from Kaggle. This synthetic data was crucial, given the scarcity of labeled fake medical image datasets. After generating 900 fake images, we combined them with the real CT scans to create a balanced binary classification dataset. This synthetic augmentation provided the necessary diversity for training the models and helped ensure that the trained models could distinguish between real and fake CT scans.

The performance of various architectures, both pretrained and custom-built, was evaluated. Pretrained models, such as VGG16, InceptionV3, and ResNet-50, demonstrated strong performance in deepfake detection, benefiting from the power of transfer learning. These models achieved high classification accuracy, with VGG16 emerging as the best performer with 98% accuracy. However, we observed that the custom-built ResNet-34 model struggled with overfitting and achieved only 50% accuracy, suggesting that further tuning and more complex architectures would be required for such models to be effective in this task.

On the other hand, custom-built CNN models specifically designed for this project also showed promising results. CNN 2 and CNN 4 achieved 98% accuracy and performed exceptionally well in real CT scan identification. These models demonstrated that, even without the benefit of transfer learning, tailored architectures and optimized hyperparameters can yield competitive performance. The use of dropout layers and careful tuning of learning rates were key factors in

achieving high generalization, reducing overfitting, and improving the models' ability to classify unseen data.

In terms of classification performance, the evaluation metrics showed that VGG16 and InceptionV3 were the most accurate models for deepfake detection, but custom CNNs like CNN 2 and CNN 4 also performed at par, especially in terms of recall for real CT scans. This is particularly important in a medical context where detecting real images accurately is paramount.

The project has successfully demonstrated the feasibility and effectiveness of using deep learning for deepfake detection in medical images, particularly lung CT scans. The results from both the pretrained and custom-built models are promising and suggest that deepfake detection in medical imaging can be implemented effectively in clinical settings.

7.2 Future Work

- **Dataset Expansion:** A larger, more diverse dataset of lung CT scans from different institutions and regions is needed to improve model robustness and generalization. Data augmentation techniques like rotation and elastic deformations should also be explored to handle variations in real-world clinical data.
- **Improved GANs:** Advanced GAN architectures, such as CycleGAN or StyleGAN, can be used to generate higher-quality synthetic images for better model training. Incorporating a more diverse set of real CT images would further enhance the generated fake images.
- **Advanced Model Architectures:** Exploring transformer-based models (like Vision Transformers) or hybrid models that combine CNNs with RNNs/LSTMs could provide more effective deepfake detection, especially for 3D scans or video data in the future.
- **Real-Time Detection:** Optimizing models for real-time detection through techniques like pruning and quantization would be essential for deployment in clinical settings. This would allow for quicker identification of fake images during routine medical imaging procedures.

References

1. Karaköse, M., Yetiş, H., & Çeçen, M. (2024). A new approach for effective medical deepfake detection in medical images. *IEEE Access*.
<https://doi.org/10.1109/ACCESS.2024.3386644>
2. Gowda, B. N., Nandeshwar, D., Raju, D. C., & Hadadi, M. S. (2024). Deep-fake detection for medical images: A survey. *International Advanced Research Journal in Science, Engineering and Technology (IARJSET)*, 11(5).
<https://doi.org/10.17148/IARJSET.2024.11595>
3. Solaiyappan, S., & Wen, Y. (2022). Machine learning based medical image deepfake detection: A comparative study. *Machine Learning with Applications*, 8, 100298.
<https://www.elsevier.com/locate/mlwa>
4. Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., ... & Bengio, Y. (2014). *Generative adversarial nets*. In *Advances in Neural Information Processing Systems* (pp. 2672–2680).
https://papers.nips.cc/paper_files/paper/2014/hash/5ca3e9b122f61f8f06494c97b1afccf3-Abstract.html
5. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep residual learning for image recognition*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 770–778). <https://doi.org/10.1109/CVPR.2016.90>
6. Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (2015). *Going deeper with convolutions*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 1–9).
<https://doi.org/10.1109/CVPR.2015.7298594>
7. Kingma, D. P., & Ba, J. (2014). *Adam: A method for stochastic optimization*. In *International Conference on Learning Representations (ICLR)*.
<https://arxiv.org/abs/1412.6980>
8. Ronneberger, O., Fischer, P., & Brox, T. (2015). *U-Net: Convolutional networks for biomedical image segmentation*. In *International Conference on Medical Image Computing and Computer-Assisted Intervention (MICCAI)* (pp. 234–241).
https://doi.org/10.1007/978-3-319-24574-4_28

Appendix I - Source Code

Method-1 (ResNet-50 Var1)

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from tensorflow.keras.optimizers import AdamW
# □ Learning rate schedule (Cosine Decay)
def cosine_decay(epoch):
    initial_lr = 0.001
    return float(initial_lr * 0.5 * (1 + tf.math.cos(epoch / 50 * 3.1416))) # Ensure float output # 50 epochs
# □ ResNet Block
def resnet_block(inputs, filters, kernel_size=3, stride=1):
    x = layers.Conv2D(filters, kernel_size, strides=stride, padding="same",
                      kernel_regularizer=regularizers.l2(1e-4))(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.Conv2D(filters, kernel_size, strides=1, padding="same",
                      kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.BatchNormalization()(x)
    shortcut = layers.Conv2D(filters, 1, strides=stride, padding="same",
                             kernel_regularizer=regularizers.l2(1e-4))(inputs)
    shortcut = layers.BatchNormalization()(shortcut)
    x = layers.Add()(x, shortcut)
    x = layers.ReLU()(x)
    return x
# □ ResNet-50 Model
def build_resnet50(input_shape=(224, 224, 3), num_classes=2):
    inputs = keras.Input(shape=input_shape)
    x = layers.Conv2D(64, 7, strides=2, padding="same", kernel_regularizer=regularizers.l2(1e-4))(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.ReLU()(x)
    x = layers.MaxPooling2D(pool_size=3, strides=2, padding="same")(x)
    # Adding ResNet blocks (50 layers total)
    filter_sizes = [64, 128, 256, 512]
    num_blocks = [3, 4, 6, 3] # Adjust to reach 50 layers
    for filters, blocks in zip(filter_sizes, num_blocks):
        for i in range(blocks):
            stride = 2 if i == 0 else 1
            x = resnet_block(x, filters, stride=stride)
    # Global Average Pooling
    x = layers.GlobalAveragePooling2D()(x)
    x = layers.Dropout(0.6)(x) # Increase dropout to avoid overfitting
    x = layers.Dense(512, activation="relu", kernel_regularizer=regularizers.l2(1e-4))(x)
    x = layers.Dropout(0.7)(x) # Higher dropout to further regularize
    outputs = layers.Dense(num_classes, activation="softmax")(x)
    model = keras.Model(inputs, outputs)
    return model
# □ Compile Model
model = build_resnet50()
model.compile(
    optimizer=AdamW(learning_rate=0.001, weight_decay=1e-4), # Replacing Lookahead + Adam
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
# □ Callbacks
early_stopping = keras.callbacks.EarlyStopping(monitor="val_loss", patience=5, restore_best_weights=True)
lr_scheduler = keras.callbacks.LearningRateScheduler(cosine_decay)
# □ Model Checkpoint (Save best model when val_loss decreases)
checkpoint_filepath = "resnet50_1_model.h5"
model_checkpoint = keras.callbacks.ModelCheckpoint(
    checkpoint_filepath, monitor="val_loss", save_best_only=True, verbose=1
)
# □ Train Model
```

```

history = model.fit(
    train_data, validation_data=val_data, epochs=50,
    callbacks=[early_stopping, lr_scheduler, model_checkpoint]
)
# □ Load Best Model & Evaluate
best_model = keras.models.load_model(checkpoint_filepath)
test_loss, test_acc = best_model.evaluate(test_data)
print(f"□ Best Model Saved at Val Loss: {min(history.history['val_loss']):.4f}")
print(f"Test Accuracy: {test_acc:.4f}")

```

Method-2 (Custom-designed CNN)

```

import numpy as np
import tensorflow as tf
import os
import json
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau, ModelCheckpoint
# □ Load preprocessed dataset
DATASET_DIR = "processed_lung_ct"
X_train = np.load(os.path.join(DATASET_DIR, "X_train.npy"))
X_val = np.load(os.path.join(DATASET_DIR, "X_val.npy"))
X_test = np.load(os.path.join(DATASET_DIR, "X_test.npy"))
y_train = np.load(os.path.join(DATASET_DIR, "y_train.npy"))
y_val = np.load(os.path.join(DATASET_DIR, "y_val.npy"))
y_test = np.load(os.path.join(DATASET_DIR, "y_test.npy"))
# □ Define CNN Model (4 Conv Layers)
model = Sequential([
    Conv2D(64, (3,3), activation='relu', padding='same', input_shape=(224, 224, 3)),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(128, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(256, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(512, (3,3), activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.5), # Prevent Overfitting
    Dense(1, activation='sigmoid') # Binary classification (Fake or Real)
])
# □ Compile Model with Learning Rate = 0.0001
model.compile(optimizer=Adam(learning_rate=0.0001), loss='binary_crossentropy', metrics=['accuracy'])
# □ Callbacks
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
lr_scheduler = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=3, verbose=1)
model_checkpoint = ModelCheckpoint("cnn_lung_ct_1_best.h5", save_best_only=True, monitor='val_loss')
# □ Train the model & save history
history = model.fit(
    X_train, y_train,
    epochs=25,
    batch_size=16,
    validation_data=(X_val, y_val),
    callbacks=[early_stopping, lr_scheduler, model_checkpoint]
)
# □ Save the final trained model
model.save("cnn_lung_ct_1.h5")

```

```
# □ Save training history to JSON
with open("training_history.json", "w") as f:
    json.dump(history.history, f)
# □ Evaluate on test set
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"□ Test Accuracy: {test_acc:.4f}")
print("□ Model training complete. Model saved as `cnn_lung_ct_1.h5` and best model as `cnn_lung_ct_1_best.h5`")
```

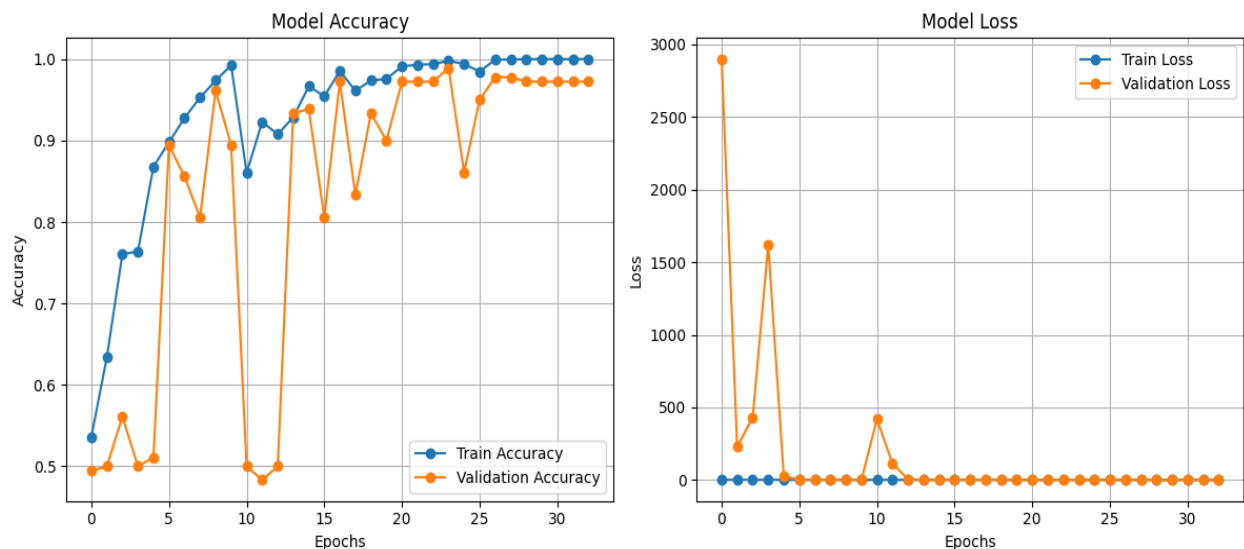
Appendix II – Screenshots

Method-1 (ResNet-50 Var1)

➤ Model Training

```
Epoch 32/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6225 - val_accuracy: 0.9722 - val_loss: 0.7279 - learning_rate: 3.4549e-04
Epoch 32: val_loss did not improve from 0.71886
Epoch 33/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6140 - val_accuracy: 0.9722 - val_loss: 0.7282 - learning_rate: 3.1594e-04
Epoch 33: val_loss did not improve from 0.71886
Epoch 34/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 34: val_loss did not improve from 0.71886
Epoch 35/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 35: val_loss did not improve from 0.71886
Epoch 36/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 36: val_loss did not improve from 0.71886
Epoch 37/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 37: val_loss did not improve from 0.71886
Epoch 38/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 38: val_loss did not improve from 0.71886
Epoch 39/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 39: val_loss did not improve from 0.71886
Epoch 40/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 40: val_loss did not improve from 0.71886
Epoch 41/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 41: val_loss did not improve from 0.71886
Epoch 42/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 42: val_loss did not improve from 0.71886
Epoch 43/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 43: val_loss did not improve from 0.71886
Epoch 44/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 44: val_loss did not improve from 0.71886
Epoch 45/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 45: val_loss did not improve from 0.71886
Epoch 46/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 46: val_loss did not improve from 0.71886
Epoch 47/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 47: val_loss did not improve from 0.71886
Epoch 48/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 48: val_loss did not improve from 0.71886
Epoch 49/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 49: val_loss did not improve from 0.71886
Epoch 50/50: 45/45 — 83s 2s/step - accuracy: 1.0000 - loss: 0.6060 - val_accuracy: 0.9722 - val_loss: 0.7309 - learning_rate: 2.8711e-04
Epoch 50: val_loss did not improve from 0.71886
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
6/6 — 4s 362ms/step - accuracy: 0.9529 - loss: 0.7565
✓ Best Model Saved at Val Loss: 0.7189
Test Accuracy: 0.9611
```

➤ Training & Validation accuracy and loss



➤ New Image prediction

```
# ✓ Example usage
IMAGE_PATH = r"C:\Users\uppal\OneDrive\Desktop\DL\f2.png" # Change this to your test image path
predict_image(IMAGE_PATH)
```

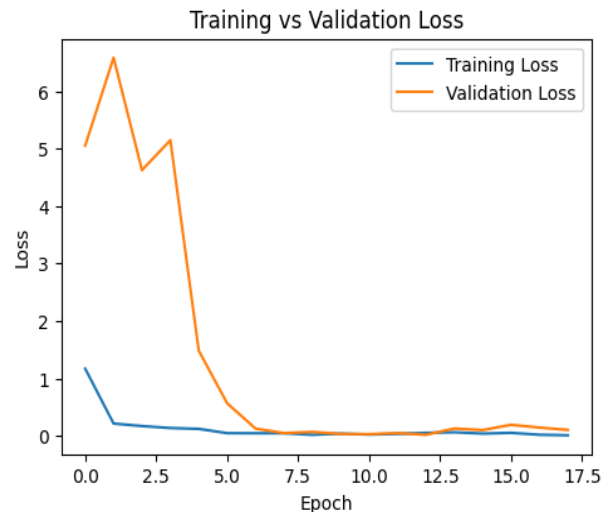
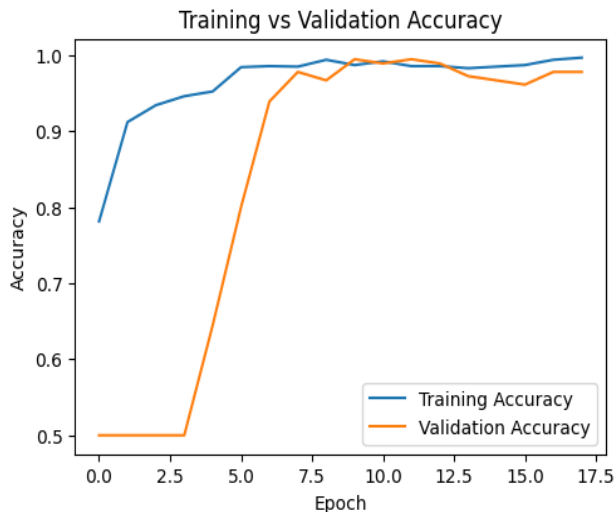
```
1/1 — 2s 2s/step
Prediction: 60.72% → ✗ Fake CT Scan (Class 1)
```

Method-2 (Custom-designed CNN)

➤ Model Training

```
Epoch 13/25 ----- 0s 3s/step - accuracy: 0.9936 - loss: 0.0242
90/90 ----- 0s 3s/step - accuracy: 0.9936 - loss: 0.0242
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
90/90 ----- 257s 3s/step - accuracy: 0.9935 - loss: 0.0245 - val_accuracy: 0.9889 - val_loss: 0.0182 - learning_rate: 1.0000e-04
Epoch 14/25 ----- 259s 3s/step - accuracy: 0.9822 - loss: 0.0486 - val_accuracy: 0.9722 - val_loss: 0.1282 - learning_rate: 1.0000e-04
Epoch 15/25 ----- 253s 3s/step - accuracy: 0.9845 - loss: 0.0467 - val_accuracy: 0.9667 - val_loss: 0.1009 - learning_rate: 1.0000e-04
Epoch 16/25 ----- 0s 3s/step - accuracy: 0.9894 - loss: 0.0344
Epoch 16: ReduceLROnPlateau reducing learning rate to 4.999999873689376e-05.
90/90 ----- 251s 3s/step - accuracy: 0.9894 - loss: 0.0346 - val_accuracy: 0.9611 - val_loss: 0.1925 - learning_rate: 1.0000e-04
Epoch 17/25 ----- 253s 3s/step - accuracy: 0.9923 - loss: 0.0275 - val_accuracy: 0.9778 - val_loss: 0.1447 - learning_rate: 5.0000e-05
Epoch 18/25 ----- 261s 3s/step - accuracy: 0.9984 - loss: 0.0070 - val_accuracy: 0.9778 - val_loss: 0.1038 - learning_rate: 5.0000e-05
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We
recommend using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
6/6 ----- 6s 918ms/step - accuracy: 0.9855 - loss: 0.0297
✓ Test Accuracy: 0.9833
✓ Model training complete. Model saved as `cnn_lung_ct_1.h5` and best model as `cnn_lung_ct_1_best.h5`
```

➤ Training & Validation accuracy and loss



➤ New Image prediction

1/1 ----- 0s 442ms/step

Prediction: Real

